

GHOSTFACE — Cryptographic Inventory

Prepared by: Ghostface Limited (NZ)
Product: GHOSTFACE — end-to-end encrypted mobile messaging app with an integrated VPN client (iOS / Android; VPN iOS-only)
Date: 19 August 2026 · **Revised:** 4 September 2026 (rev. 3 — see §6)
Purpose: Supporting document for export-control classification review. Factual inventory of all cryptography used in the product, its sources, and its purpose. Prepared for review by legal counsel — this document states technical facts, not legal conclusions.

1. Summary

GHOSTFACE is a consumer end-to-end encrypted (E2EE) messaging and calling application, freely available to the general public through the Apple App Store and Google Play. Its messaging encryption exists to protect users’ personal communications in transit and at rest.

The application **also includes a VPN client built on WireGuard** (iOS only), offered as a user-facing feature with a choice of server locations. The VPN carries the device’s network traffic through an encrypted tunnel and is therefore a second, separate cryptographic subsystem with its own protocol and its own implementation. It is described at §3, *WireGuard VPN tunnel*. **This was omitted from revisions 1 and 2 in error; see §6.**

Every cryptographic algorithm used is a **standard, published algorithm** drawn from NIST FIPS standards, IETF RFCs, and peer-reviewed public specifications. The messaging cryptography is implemented using widely-used open-source libraries — the @noble/* JavaScript libraries for the protocol primitives, and react-native-argon2 for the Argon2id recovery-PIN derivation — together with the operating system’s own cryptographic random-number and key-storage facilities. The VPN cryptography is the upstream WireGuard implementation, vendored unmodified as C, Swift and Go sources (§3, *WireGuard VPN tunnel*). **No proprietary or secret cryptographic algorithm has been designed or implemented in either subsystem.**

The protocol design follows the publicly-documented Signal protocol family (X3DH + Double Ratchet), extended for post-quantum security using the same hybrid approach published by Signal (PQXDH) and Apple (iMessage PQ3).

2. Cryptographic primitives

Function	Algorithm	Standard	Source library (version)
Authenticated encryption (messages, media, storage)	ChaCha20-Poly1305	RFC 8439	@noble/ciphers 2.1.1
Key agreement (classical)	X25519 (ECDH on Curve25519)	RFC 7748	@noble/curves 2.0.1
Digital signatures	Ed25519	RFC 8032	@noble/curves 2.0.1
Post-quantum key encapsulation	ML-KEM-768	NIST FIPS 203	@noble/post-quantum 0.6.1

Function	Algorithm	Standard	Source library (version)
Hashing	SHA-256, SHA-512	FIPS 180-4	@noble/hashes 2.0.1
Key derivation (session)	HKDF-HMAC-SHA256	RFC 5869	@noble/hashes 2.0.1
Key derivation (PIN/passphrase)	PBKDF2-HMAC-SHA256, 600,000 iterations	RFC 2898 / NIST SP 800-132	@noble/hashes 2.0.1
Key derivation (recovery PIN)	Argon2id — t=3, m=64 MiB, p=1, 32-byte output	RFC 9106	react-native-argon2 4.0.0 (native)
Message authentication	HMAC-SHA256	RFC 2104 / FIPS 198-1	@noble/hashes 2.0.1
Random number generation	OS CSPRNG — SecRandomCopyBytes (iOS), java.security.SecureRandom (Android)	Platform-provided	react-native-get-random-values 1.11.0, expo-crypto
Key storage	iOS Keychain / Android Keystore	Platform-provided	expo-secure-store
Tunnel key agreement (VPN)	Curve25519 ECDH	RFC 7748	vendored WireGuardKitC/x25519.c (MIT, J. A. Donenfeld; TweetNaCl-derived)
Tunnel handshake + transport (VPN)	Noise_IKpsk2 — ChaCha20-Poly1305, BLAKE2s, HKDF	WireGuard protocol specification; RFC 8439, RFC 7693	vendored golang.zx2c4.com/wireguard (2023-02-09), golang.org/x/crypto 0.6.0

All of the above are standard published algorithms. None is modified.

3. Protocols and how the primitives are combined

GHOSTFACE composes the standard primitives above into the publicly-documented Signal protocol design:

Initial key agreement — X3DH, extended to PQXDH. New sessions are established using Signal's Extended Triple Diffie-Hellman (X3DH) over X25519, with prekey signatures verified via Ed25519. Where both parties support it, the handshake additionally performs an ML-KEM-768 key encapsulation and mixes the resulting shared secret into the session key — identical in structure to Signal's published PQXDH specification. The app requires this post-quantum step for all new sessions.

Ongoing messaging — Double Ratchet. Established sessions use Signal's Double Ratchet algorithm: a Diffie-Hellman ratchet (X25519) plus symmetric-key ratchets, with per-message keys deriving forward secrecy and post-compromise security. Each message is encrypted with ChaCha20-Poly1305 under a per-message key.

Key-derivation functions. Root- and chain-key derivation uses HKDF-HMAC-SHA256 (RFC 5869) throughout — the standard construction used by the Signal protocol itself.

Post-quantum continuous rekey (kdfrkPQ). For PQ-enabled sessions, the root-key update mixes the post-quantum shared secret into the ratchet. This is implemented as a single HKDF-HMAC-SHA256 call whose input keying material is the concatenation of (a) the X25519 Diffie-Hellman output and (b) the ML-

KEM-768 shared secret, with a distinct domain-separation label. **It is standard HKDF-SHA256 applied to two standard shared secrets — it defines no new cryptographic algorithm or primitive.** Its purpose and structure mirror the continuous post-quantum rekeying published in Apple’s iMessage PQ3 design.

Data at rest. Locally-stored data (message history, keys) is encrypted with ChaCha20-Poly1305 under a master key held in the OS keychain (iOS Keychain / Android Keystore). PIN/passphrase-derived keys use PBKDF2-HMAC-SHA256 at 600,000 iterations (OWASP-recommended).

Recovery-PIN key blinding (Argon2id). The 24-word recovery phrase is a reversible BIP39 encoding of the identity private key, so a leaked phrase alone would be total compromise. To prevent that, the key is blinded with a user-chosen 6-digit PIN before the phrase is generated:

```
phraseValue = identityKey XOR Argon2id(pin, salt)
```

- **Purpose:** make the recovery phrase useless on its own — restore requires both the phrase and the PIN. The PIN is never stored or transmitted.
- **Inputs:** the 6-digit PIN, and a per-identity salt derived deterministically as SHA-256 of a fixed domain-separation tag concatenated with the normalised alias. The salt is public and derivable by design: a salt requires uniqueness, not secrecy, and deriving it from the alias keeps the feature entirely client-side with nothing stored server-side.
- **Parameters:** Argon2id, 3 iterations, 64 MiB memory, parallelism 1, 32-byte output — chosen because an attacker holding the phrase can brute-force the PIN offline against the publicly-fetchable identity key, so each guess must be made expensive. Run in native code (react-native-argon2) as pure-JS Argon2 is prohibitively slow on the mobile JavaScript engine.
- **Output:** a 32-byte blinding value, combined with the identity private key by bitwise XOR.

It is the standard Argon2id function of RFC 9106, used conventionally as a password-based key-derivation function. It defines no new cryptographic algorithm or primitive, and performs no encryption — the derived value is used only to blind a key at rest on the user’s own device.

Implemented in `artifacts/ghostface/lib/recoveryPin.ts`.

WireGuard VPN tunnel (added rev. 3)

In addition to the messaging protocol, the application includes a **VPN client** built on WireGuard. It is a user-facing feature: a top-level “VPN” tab offers a choice of six server locations (United States, Germany, Japan, Sweden, Iceland, Singapore) with connect/disconnect control.

Implementation. The upstream wireguard-apple sources are vendored into the repository at `artifacts/ghostface/native/wireguard-apple/`. The tunnel runs in a separate iOS Network Extension process (entitlement `com.apple.developer.networking.networkextension`) driven via `NETunnelProviderManager`. This code is **not** part of the `@noble/*` JavaScript stack used by the messaging protocol — it is upstream C, Swift and Go compiled into the application.

Cryptography. Standard WireGuard, unmodified: the Noise_IKpsk2 handshake pattern with Curve25519 ECDH, ChaCha20-Poly1305 for transport authenticated encryption, BLAKE2s for hashing and MAC, and HKDF for key derivation. The optional pre-shared-key slot is not used — the tunnel configuration surface (`artifacts/ghostface/lib/vpnTunnelModule.ts`) exposes no pre-shared key field. Curve25519 key

material for the extension is generated using the operating system's RNG (`CommonCrypto/CommonRandom.h`).

Scope of traffic. The default allowedIPs is `0.0.0.0/0,::/0` — a full tunnel carrying all of the device's network traffic, not only the application's own traffic.

Platforms. iOS only. Android has no equivalent module.

No modification to the cryptography. The vendored sources are upstream. The only local change to any file in this component is a compile-time include added to `Sources/WireGuardKitC/WireGuardKitC.h` — recorded at §5.

4. Nature and availability

- **Function:** data confidentiality and integrity — for personal communications (messaging subsystem), and for the device's network traffic in transit while the VPN tunnel is active (§3, *WireGuard VPN tunnel*).
 - **Availability:** free consumer app, distributed to the general public through mainstream app stores; no restriction on class of user.
 - **Cryptographic novelty:** none at the primitive level. The only non-verbatim element (`kdfRkPQ`) is a standard-HKDF composition of two standard shared secrets, matching published hybrid-PQ designs. Argon2id and the recovery-PIN blinding use the published function unmodified.
 - **Key length / strength:** as specified by the underlying standards (e.g. 256-bit symmetric, Curve25519, ML-KEM-768 / NIST security category 3).
 - **Open-source dependencies:** all cryptography via public open-source libraries — `@noble/*` for the messaging protocol primitives, `react-native-argon2` for Argon2id, and the upstream WireGuard implementation (`golang.zx2c4.com/wireguard`, `golang.org/x/crypto`, `WireGuardKitC`) for the VPN tunnel; the app implements no cryptography in a closed or secret manner.
-

5. Implementation corrections (no change to algorithms or protocol)

This section records changes that correct **how** existing cryptography is implemented, without altering which algorithms are used, how the protocol is structured, or what the cryptography does.

Canonical encoding of AEAD associated data (Double Ratchet message headers).

Each Double Ratchet message is bound to its header by passing the header as associated data (AD) to ChaCha20-Poly1305. The AD was previously produced by serialising the header object to JSON. That is deterministic in practice for the present single implementation, but it is not a sound long-term invariant: a second implementation in another language, or an incidental change that reordered object keys, could produce a different AD for what should be an identical header.

The correction replaces JSON serialisation with a fixed, self-describing binary encoding: a constant magic value and protocol-version byte, a fixed field order and field count, fixed integer widths in big-endian order, raw key bytes rather than hex text, and explicit presence-and-length framing for the two optional post-quantum fields so that a field is never simply omitted. A single function produces the AD for both encryption and decryption.

- **Algorithms:** unchanged — ChaCha20-Poly1305 (RFC 8439), as listed in §2.

- **Protocol:** unchanged — Double Ratchet header fields and their meaning are the same; only their byte-level representation as AEAD associated data changes.
- **Functionality:** unchanged — the AD continues to bind each message to its header, as before.
- **Nature:** encoding hygiene and cross-implementation robustness. No known exploit exists against the previous encoding; this is a correctness and future-proofing fix, not a patched vulnerability.

Implemented in `artifacts/ghostface/lib/doubleRatchet.ts`.

Missing system include in the vendored WireGuard C header.

`native/wireguard-apple/Sources/WireGuardKitC/WireGuardKitC.h` gains a single `#include <sys/types.h>`. The header redeclares `struct ctl_info` and `struct sockaddr_ctl` from `<sys/kern_control.h>` using the BSD `u_int32_t`, `u_char` and `u_int16_t` typedefs, which the upstream source relies on arriving implicitly. Under explicit Clang modules the compiler rejects this, and the `WireGuardKitC` module fails to build.

- **Algorithms:** unchanged — Curve25519, ChaCha20-Poly1305, BLAKE2s and HKDF as listed in §2, all as implemented upstream.
- **Protocol:** unchanged — the Noise_IKpsk2 handshake and WireGuard transport are untouched.
- **Functionality:** unchanged — the emitted code is the same; the include affects type visibility at compile time only.
- **Nature:** a build correction. Without it the component does not compile at all under a toolchain that enforces explicit modules.

Applied to the vendored copy at `artifacts/ghostface/native/wireguard-apple/Sources/WireGuardKitC/WireGuardKitC.h`. It is the only local modification to that vendored component.

6. Change record

Changes since the 19 August 2026 version (rev. 1 → rev. 2)

Both items below were raised with counsel on 30 August 2026 and considered in counsel's response of **31 August 2026**, which confirmed that neither changes the classification analysis or the conclusions in the advice memorandum, and recommended that this inventory be updated to record them.

#	Item	Where	Nature
1	Argon2id added — recovery-PIN key blinding	§2 table, §3	Addition of a published, conventionally-applied key-derivation function (RFC 9106). Present in the codebase from 27 August 2026; omitted from rev. 1 in error.
2	AEAD associated-data canonical encoding	§5	Implementation correction only. Algorithms, protocol and functionality unchanged.

Neither item introduces a proprietary or novel cryptographic algorithm, and neither changes the set of primitives in §2 other than by recording Argon2id, which was already in use when rev. 1 was prepared.

Changes since the 31 August 2026 version (rev. 2 → rev. 3)

⚠ Unlike the rev. 2 items above, these have NOT yet been put to counsel. They are recorded here first so that the description is accurate before they are raised.

#	Item	Where	Nature
3	WireGuard VPN tunnel recorded — previously absent	§1, §2 table, §3, <i>WireGuard VPN tunnel</i> , §4	Addition to the <i>description</i> , not to the codebase. The VPN is a shipped, user- facing feature and was omitted from revisions 1 and 2 in error. It is a second cryptographic subsystem: Noise_IKpsk2 with Curve25519, ChaCha20- Poly1305, BLAKE2s and HKDF, implemented by the upstream WireGuard sources vendored unmodified. Default allowedIPs 0.0.0.0/0, ::/0 carries all device traffic.
4	#include <sys/ types.h> added to WireGuardKitC.h	§5	Compile-time correction only. The header redeclares struct ctl_info/ struct sockaddr_ctl using BSD u_* typedefs that upstream relies on arriving implicitly; explicit Clang modules require the include. No algorithm, protocol, key handling, parameter or runtime behaviour changes.

Item 3 is a correction to the completeness of this inventory rather than a change in the product: the functionality it describes was present when revisions 1 and 2 were prepared. Item 4 introduces no proprietary or novel cryptographic algorithm and does not change the set of primitives in §2.

This inventory reflects the state of the codebase as of 31 August 2026 and is provided to assist counsel's export-control classification assessment. It is a technical description prepared by the company, not a legal classification. Revision 1 (19 August 2026) described the codebase as at that date; the changes in revisions 2 and 3 are itemised in §6. Revision 3 (4 September 2026) records the WireGuard VPN subsystem, which was present but undescribed in revisions 1 and 2.